

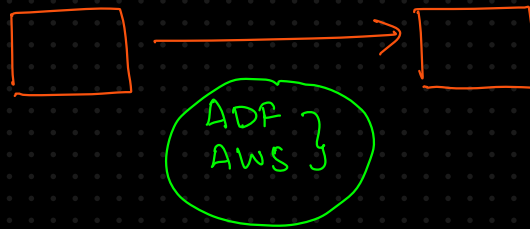
Airflow

Cron Jobs

Scripts for use cases

Scripts

as the use case grow
handling cron jobs / AWS glue
jobs etc. becomes difficult



airflow is an orchestration tool



Apache Airflow is an open source platform to programmatically author,
schedule and monitor workflows.

Code first → python code

Extensible → hundred of operators

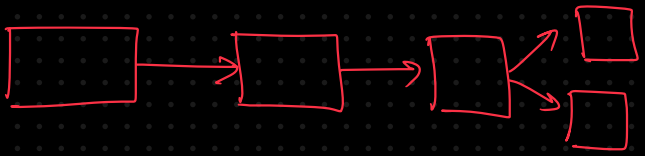
Scalable → single / multi node

DAAG

Airflow → reads
postgres SQL

PC → monitor
ram
harddisk

Term	Definition
DAG	A Directed Acyclic Graph: the blueprint of your workflow, declaring tasks & dependencies.
Task	One node in the DAG, a single unit of work (e.g., run SQL, execute Python, trigger Spark).
Operator	Abstracts how a task runs: BashOperator, PythonOperator, SparkSubmitOperator, etc.
Task Instance	A specific run of a task, tied to a DAG run and execution date.
DAG Run	One instantiation of the entire DAG at a given schedule timestamp (e.g., "2025-05-24").
Scheduler	Component that parses DAG files and enqueues task instances when their schedule arrives.
Executor	Decides how tasks are executed (locally, via Celery, Kubernetes, etc.).
Web Server	Hosts the UI (React-based in v3.x) for monitoring and interaction.
Metastore	A relational database (Postgres/MySQL) storing DAGs, task states, logs, configs, etc.
Worker	Process (or pod/container) that actually executes task code (used in distributed modes).



No SQL Database



not only SQL

A category of dBs designed for
flexible, scalable data storage & access
→ without schema instructions

creating data

traditional db



noSQL

Feature	SQL (Relational)	NoSQL (Non-Relational)
Schema	Fixed (tables, columns)	Dynamic / schema-less
Data Relationships	Strong with JOINS	Typically denormalized
Scalability	Vertical (add RAM/CPU)	Horizontal (add more machines)
Query Language	SQL	Varies (JSON-like, key-value, etc.)
Ideal For	Structured data & complex queries	Big data, real-time, flexible models

No SQL architecture — a high level overview

Shared architecture

each node has data / shares data

Nodes can be added without downtime.

Sharding: splitting data across nodes

Replication:

Eventual Consistency:

Scheduler

Cron expression

Cron expression generator by [Cronhub](#). Schedule and monitor jobs without any infra work.

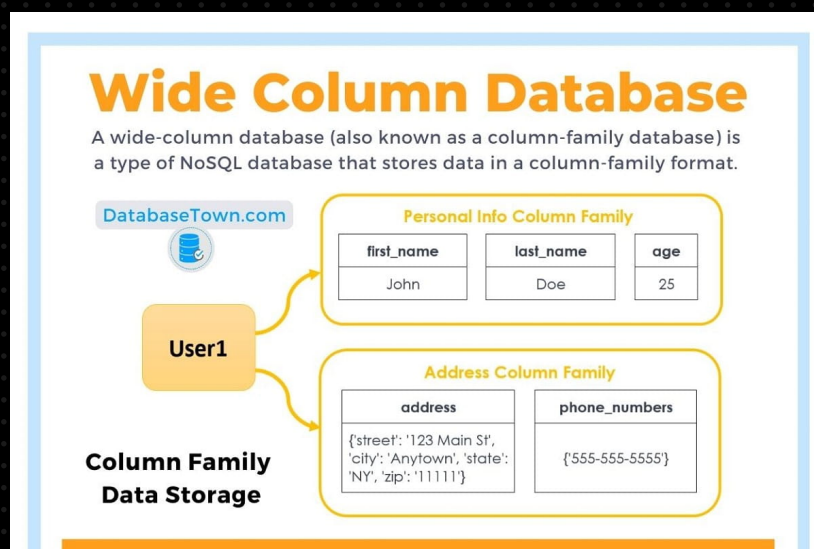
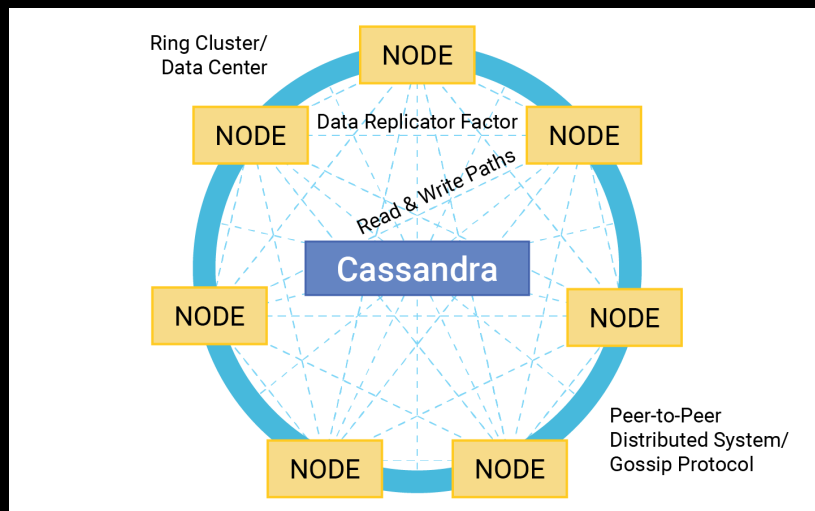
Every minute

The cron expression is made of five fields. Each field can have the following values.

*	*	*	*	*
minute (0-59)	hour (0 - 23)	day of the month (1 - 31)	month (1 - 12)	day of the week (0 - 6)

Here are some examples for you.

Cron expression	Schedule
*****	Every minute
0*****	Every hour
00****	Every day at 12:00 AM
00**FRI	At 12:00 AM, only on Friday
001**	At 12:00 AM, on day 1 of the month



Cassandra DB ← python

Data Modelling & System design

Not in Syllabus,
extra topic being
covered.

way to structure our data so that we can store & define
it in a very optimized manner

OLTP ✓
DB

OLAP ✓
DWH



data
analyst

data
engineer

easy to write
Query with
usability

highly optimized Query
Complexity is fine

Data Modelling → model our data pretty well
⇒ reduce redundancy

OLTP → DB
write / update optimized

Normalization — dividing big table into multiple smaller tables.

For OLTP, we need normalization so as to make it good for Quick
reads and write

⇒ less redundancy

⇓

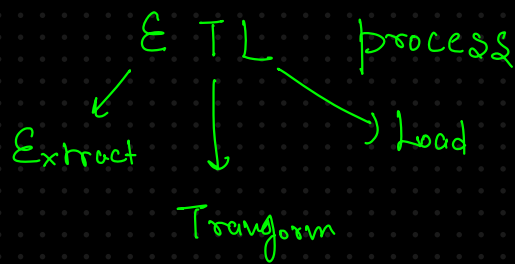
less writes

⇓

Quicker updates

for reporting requirements can be different.

OLAP systems



DWH systems

analytical queries

read heavy

& writes are less / none.

Data modelling in DWH

→ design a technique so as to support end user queries

Process of modelling a business process into series of facts & dimⁿ tables for analysis

Transaction DB

insert & update Quick

data consistency is critical

normalization

focus on fast maintenance of data

Reporting DB (DWH)

optimized for running queries.

not worried about maintenance of data

Query → analytics

insert & update, they are not very relevant

Fact



measurable metric

Dimension



enhances your fact data

Amazon → Rs 5000

5000



Mayank
amazon.shopping@gmail
add. _____

fact

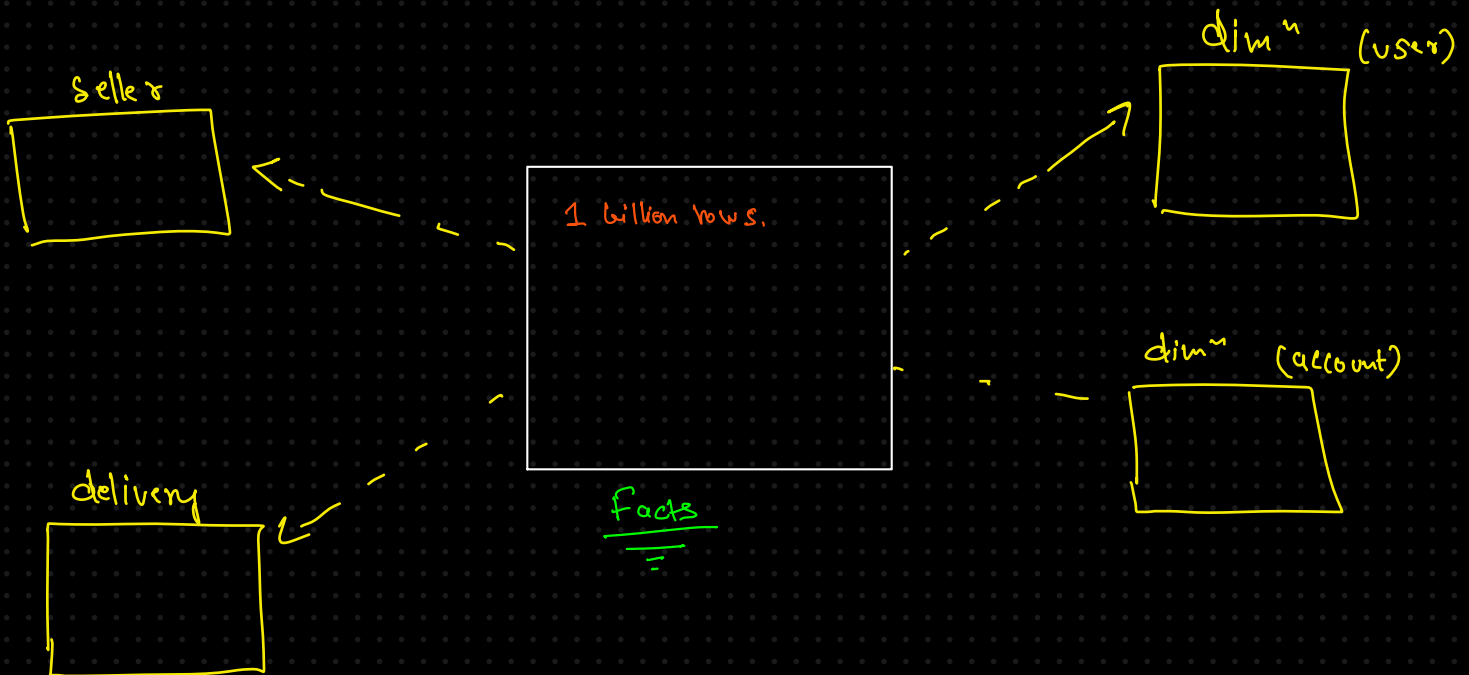
\$ iPhone → \$ 700 usd

Dimension → enhances the facts

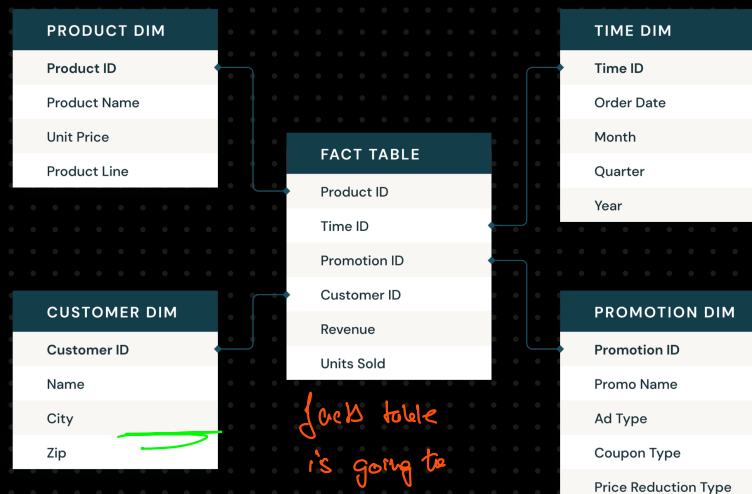
mayank
mayank@amazon.com
Delhi

Type of delivery
Account - Prime or not

Product
Shipper / Seller



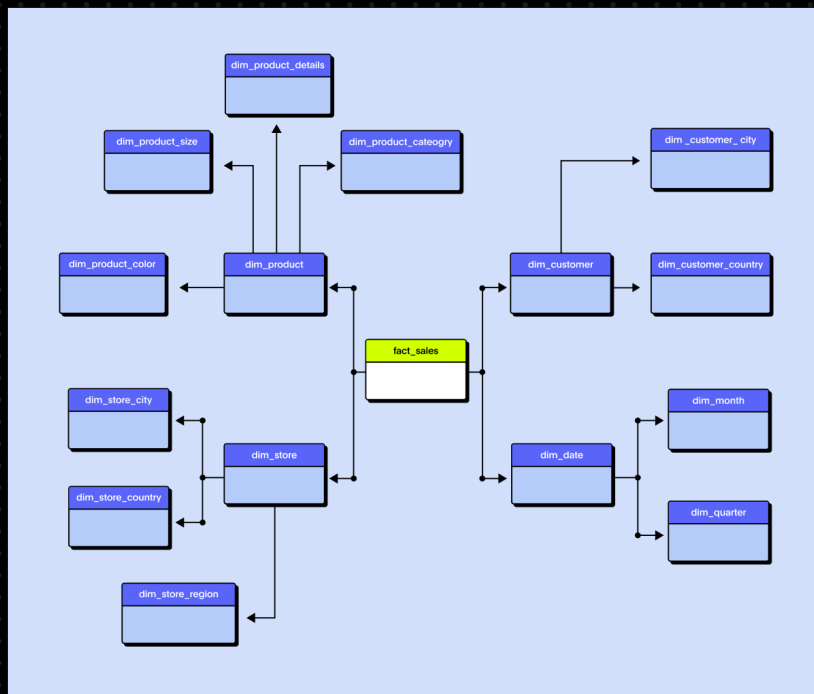
Star schema



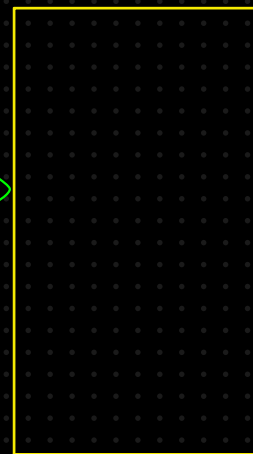
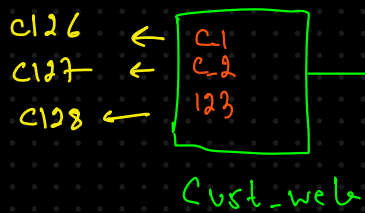
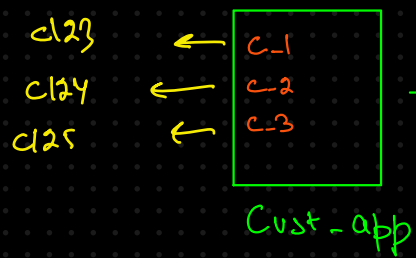
facts table
is going to
be large not
data

1. Dimension are denormalized.

2. How dimensions are not connected to one another



Surrogate Key



all customers

duplicates
 corrupted keys
 different data
 types

Steps for Dimension Modelling

1. Identify/Choose the business process
2. Define/declare the Grain (Granularity)

order → 5 items → total items
 → 200 100 300 250 150 → price of ind items
 → \$1000 → cost total

3. Identify the dimⁿ
 4. Identify the facts
- data understanding

SCD → Slowly changing dimension

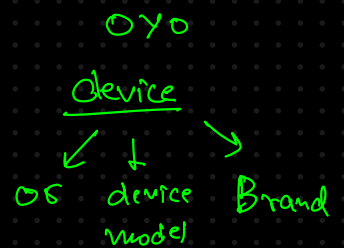
500

SCD 0 ⇒ never changes (DOB,
 Place of Birth,
 Registration date
 Blood Marks)

-100

SCD 1 ⇒ we will overwrite
 (no history)

Device
~~Android~~ → iPhone
 iPhone



email & mobile no. ✓

iPhone ⇒ 80%

SCD-2 → maintain full history

device	start_date	end_date
Android	2016	2019
iPhone	2020	Present/future

2019 → 60% android

2020 → 70% iPhone

SCD-3 → maintain last value / history / Partial history

old_device new_device

Rapid changing data / dimⁿ

Snapshot → daily
monthly

